

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA152 Microcontrollers

Last Name: _____ Date: _____
First Name: _____ Subject/Section: _____

Laboratory Activity No. 1
Real-Time Multisensor Room Monitoring System
100 Points

1. Laboratory Objective

This laboratory requires students to design and implement, from scratch, a simulated ESP32 room-monitoring system. Students must reconstruct the system requirements, redesign the system for Wokwi, implement a FreeRTOS-based architecture, verify the firmware, document development decisions, and publish the finished work as part of a personal technical portfolio.

2. Learning Outcomes

At the end of this laboratory activity, the students should be able to:

1. Create an ESP32 project using PlatformIO.
2. Construct and simulate a microcontroller circuit in Wokwi.
3. Interface sensors and actuators with an ESP32.
4. Organize firmware into multiple source modules.
5. Create and manage FreeRTOS tasks.
6. Assign and justify task priorities.
7. Distinguish Running, Ready, Blocked, Suspended, and Deleted task states.
8. Use queues for inter-task communication.
9. Use a mutex to protect a shared resource.
10. Use an event group or task notification for event signaling.
11. Implement periodic execution using `vTaskDelayUntil()`.
12. Implement a simple embedded-system state machine.
13. Separate hardware-independent decision logic from hardware drivers.
14. Write and execute automated unit tests using PlatformIO.
15. Perform static code analysis using PlatformIO.
16. Use Git incrementally and maintain a professional GitHub repository.
17. Document a project for both academic assessment and public portfolio use.
18. Communicate the completed project through Hackster.io.

3. System to Develop

Students will create a FreeRTOS-Based ESP32 Room Multisensor. The simulated device shall monitor room/environmental conditions and provide user interaction using components available in Wokwi.

Component	Purpose
ESP32	Main microcontroller
DHT22	Temperature and humidity
Photoresistor / LDR	Ambient light
PIR sensor	Motion detection
Rotary encoder	User input
SSD1306 OLED	Information display
Buzzer	Alarm output

4. Functional Requirements

ID	Requirement	Description
FR-01	Temperature measurement	The system shall periodically obtain temperature data.
FR-02	Humidity measurement	The system shall periodically obtain humidity data.
FR-03	Ambient-light measurement	The system shall monitor relative ambient-light level.
FR-04	Motion detection	The system shall detect simulated motion using the PIR sensor.
FR-05	OLED display	The system shall display one selected measurement at a time.
FR-06	Rotary encoder navigation	The rotary encoder shall switch among Temperature, Humidity, Light, and Motion.
FR-07	Temperature alarm	The buzzer shall activate when temperature is outside the configured normal range.
FR-08	Activity state	The system shall support ACTIVE and INACTIVE states.
FR-09	Automatic inactivity	If no motion is detected for a specified duration, the system shall enter INACTIVE.
FR-10	Automatic reactivation	Motion shall return the system to ACTIVE.

For this laboratory, use the following temperature limits: (i) LOW TEMPERATURE LIMIT = 18 °C and (ii) HIGH TEMPERATURE LIMIT = 30 °C.

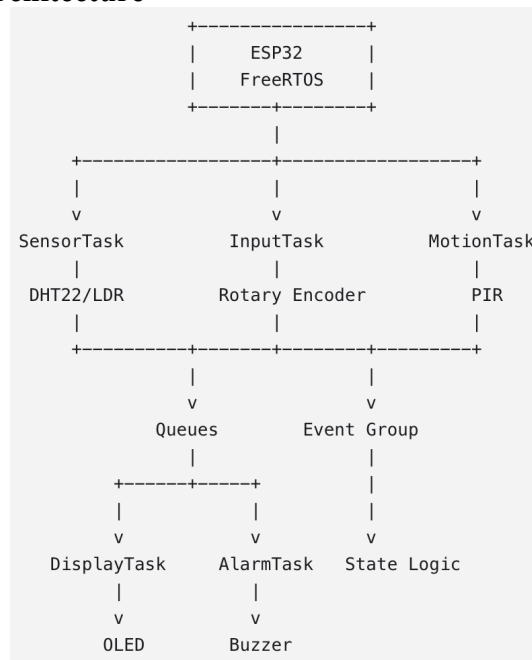
6. FreeRTOS Is Mandatory

This laboratory shall be implemented exclusively using PlatformIO with the Espressif IoT Development Framework (ESP-IDF), using C and/or C++ and the native FreeRTOS APIs provided by ESP-IDF. The Arduino framework and Arduino-specific abstractions are prohibited. The application shall be designed as a concurrent embedded system. A submission that places nearly all application behavior in a single task or otherwise avoids the required FreeRTOS architecture will be treated as an incomplete submission, hence, will automatically be given 0 points.

7. Minimum Required Tasks

Task	Primary Responsibility
SensorTask	Read DHT22 and LDR periodically.
DisplayTask	Own and manage the OLED.
InputTask	Process rotary-encoder navigation.
MotionTask	Monitor PIR activity.
AlarmTask	Evaluate alarm state and control the buzzer.
StateTask (recommended)	Centralize ACTIVE/INACTIVE state management.

8. Expected FreeRTOS Architecture



The actual architecture may differ, but every deviation must be technically defensible and consistent with the implemented code.

9. Required FreeRTOS Concepts

- Multiple tasks
- Explicit task priorities
- Blocking delays
- `vTaskDelayUntil()` for at least one periodic task
- At least one FreeRTOS queue
- At least one mutex
- At least one event group or task notification
- A state machine
- Inter-task communication
- Shared-resource protection

Merely declaring a queue, mutex, or event group to satisfy a checklist earns no credit. Each mechanism must have a legitimate role in the implemented design.

10. Development Environment Verification

Before starting, verify the required development tools. Required firmware environment: ESP-IDF with C/C++. Use `app_main()` as the application entry point and native ESP-IDF/FreeRTOS APIs for GPIO, ADC, I2C, logging, tasks, queues, mutexes, event groups, and timing.

- `git --version`
- `pio --version`

If either command fails, resolve the environment before beginning implementation.

PART I — Project Initialization

11. Create the GitHub Repository

Create a public GitHub repository. Recommended name: `bca152-freertos-multisensor`. Avoid vague portfolio repository names such as `lab5`, `final`, `activity`, or `project123`.

12. Clone the Repository

- `git clone https://github.com/YOUR_USERNAME/bca152-freertos-multisensor.git`
- `cd bca152-freertos-multisensor`

13. Create the PlatformIO Project

1. Open PlatformIO Home in Visual Studio Code.
2. Create a New Project.
3. Choose an ESP32 development board appropriate for Wokwi.
4. Select ESP-IDF as the framework.
5. Create the project inside the GitHub repository.

The PlatformIO configuration must identify ESP-IDF explicitly. A minimum configuration is:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
monitor_speed = 115200
```

Use of framework = arduino, Arduino-only libraries, or Arduino-specific APIs is not permitted for this laboratory.

Expected repository structure:

```
bca152-freertos-multisensor/
├── include/
├── lib/
├── src/
├── test/
├── docs/
├── platformio.ini
├── diagram.json
├── wokwi.toml
└── README.md
```

14. First Build

Compile the empty project before adding sensors or FreeRTOS application logic.

- pio run

Do not proceed until the base project compiles successfully.

15. First Commit

- git add .
- git commit -m "Initialize ESP32 PlatformIO project"
- git push

This commit must occur before substantial implementation begins.

PART II — Wokwi Simulation

16. Create the Initial Wokwi Circuit

Start with the ESP32 only. Confirm that the firmware starts in Wokwi and produces a recognizable serial message.

- BCA152 FreeRTOS Multisensor
- System starting...

Commit this milestone with a meaningful message such as “Configure initial Wokwi simulation.”

PART III — FreeRTOS Foundation

17. Create Two Simple Tasks First

Before integrating sensors, create two simple FreeRTOS tasks that periodically print distinct diagnostic messages. Both tasks must block between executions.

- Task A running
- Task B running
- Task A running
- Task B running

18. Observe Scheduling and Task States

Record assigned priority, delay interval, execution frequency, and what occurs while a task is blocked. Be prepared to distinguish Running, Ready, and Blocked states in your own implementation.

- SensorTask executing -> Running
- SensorTask waiting 2 sec -> Blocked
- SensorTask waiting for CPU -> Ready

19. Do Not Write Uncontrolled Busy Loops

A continuously executing task that never blocks can monopolize processor time and interfere with lower-priority work. Each continuously running task must perform finite work and then block, wait, or yield appropriately.

PART IV — Sensor Subsystem

20. Add the DHT22

Connect the simulated DHT22. Verify temperature and humidity first through the Serial Monitor before integrating the OLED or other tasks.

- Temperature: 25.40 C
- Humidity: 61.20 %

21. Add the LDR

Read the photoresistor through an ESP32 ADC input. Convert the raw reading into a documented representation such as 0–100%. Do not claim calibrated lux unless the conversion is implemented and justified.

22. Create SensorTask

Move sensor acquisition into a FreeRTOS task. Use periodic execution. At least one periodic task in the laboratory must use `vTaskDelayUntil()`.

```
TickType_t lastWakeTime = xTaskGetTickCount();

for (;;) {
    readSensors();
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(2000));
}
```

23. Required Explanation: `vTaskDelayUntil()`

In the laboratory report, explain why `vTaskDelayUntil()` is generally more appropriate than `vTaskDelay()` for periodic sensor sampling. A correct explanation must address periodic timing and drift, not merely state that the function “creates a delay.”

PART V — Data Communication

24. Define Sensor Data

```
struct SensorData {
    float temperature;
    float humidity;
    int lightLevel;
    bool motionDetected;
};
```

25. Create a Queue

Use a FreeRTOS queue to communicate sensor information to one or more consumers. Avoid unsynchronized global variables as the default design.

```
SensorTask
|
v
Sensor Queue
|
+-----> DisplayTask
|
+-----> AlarmTask
```

PART VI — Display Subsystem

26. Add the OLED

The OLED shall be owned by DisplayTask. Multiple tasks shall not write directly to the OLED unless the student can justify and correctly synchronize that design.

27. Initial OLED Output

```
ROOM MONITOR

Temperature
25.4 C
```

Once this basic display works reliably, proceed to user navigation.

PART VII — Rotary Encoder

28. Add the Encoder and InputTask

Create InputTask. Use an enumeration for the displayed page rather than arbitrary numeric constants.

```
enum class DisplayMode {
    TEMPERATURE,
    HUMIDITY,
    LIGHT,
    MOTION
};
```

29. Navigation Behavior

```
Clockwise:    Temperature -> Humidity -> Light -> Motion
-> Temperature
```

Counterclockwise: reverse traversal, including wraparound

PART VIII — Alarm Subsystem

30. Create Testable Alarm Logic

Separate temperature decision logic from buzzer hardware control. Pure decision logic can be unit tested independently of Wokwi and the ESP32.

```
enum class AlarmState {
    NORMAL,
    LOW_TEMPERATURE,
    HIGH_TEMPERATURE
};
```

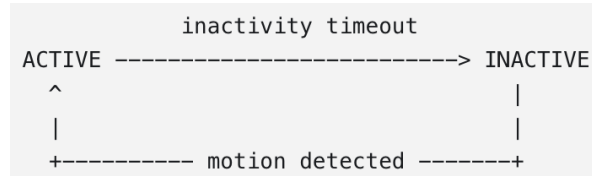
```
AlarmState evaluateTemperature(float temperature);
```

PART IX — Motion and System State

31. Add the PIR and MotionTask

Create MotionTask to monitor motion. For laboratory testing, use a short inactivity timeout such as 15 seconds.

32. Implement the State Machine



33. ACTIVE Behavior

- OLED enabled.
- Normal sensor processing.
- Encoder active.
- Alarm active.

34. INACTIVE Behavior

- OLED off or blank.
- Unnecessary display operations reduced.
- Motion detection remains operational.
- Subsequent PIR activity restores ACTIVE.

PART X — Event Group or Task Notification

35. Use an Event Signaling Mechanism

Use an event group or task notification for meaningful system events. If an event group is selected, document each bit, its producer, its consumer, and when it is set or cleared.

```
#define EVENT_ACTIVE BIT0
#define EVENT_MOTION BIT1
#define EVENT_ALARM BIT2
```

PART XI — Mutex

36. Protect a Shared Resource

A suitable beginner example is shared Serial output. If multiple tasks print diagnostics, use a mutex to prevent interleaved writes.

```
xSemaphoreTake(serialMutex, portMAX_DELAY);
// Serial output
xSemaphoreGive(serialMutex);
```

37. Required Mutex Explanation

The laboratory report must identify the actual shared resource, the competing tasks, and the failure mode that the mutex prevents. “A mutex prevents race conditions” by itself is insufficient.

PART XII — Task Priorities

38. Assign Explicit Priorities

Task	Suggested Starting Priority
MotionTask	3
InputTask	3
SensorTask	2
AlarmTask	2
DisplayTask	1

Students may choose different priorities, but must justify them in terms of scheduling urgency and acceptable latency.

39. Priority Is Scheduling Urgency

Avoid explanations such as “MotionTask has priority 3 because it is important.” Instead explain which tasks require prompt response, which can tolerate latency, and what could happen if priorities are poorly selected.

PART XIII — Modular Software Design

40. Required Source Organization

```
include/
├── sensors.h
├── display.h
└── input.h
```

```
|— alarm.h
|— motion.h
|— system_state.h
|— rtos_objects.h

src/
|— main.cpp
|— sensors.cpp
|— display.cpp
|— input.cpp
|— alarm.cpp
|— motion.cpp
|— system_state.cpp
|— rtos_objects.cpp
```

Exact organization may differ if technically justified. The entire application shall not be placed in main.cpp.

41. Keep main.cpp Focused

```
hardware initialization
      ↓
FreeRTOS object creation
      ↓
task creation
      ↓
scheduler-driven operation
```

PART XIV — Unit Testing

42. What to Unit Test

Begin with deterministic, hardware-independent application logic. Recommended targets include:

```
evaluateTemperature()
nextDisplayMode()
previousDisplayMode()
evaluateSystemState()
```

43. Minimum Unit Tests

Category	Minimum	Required Coverage
Temperature alarm logic	5	Below lower threshold; exactly lower threshold; normal value; exactly upper threshold; above upper threshold.
Display navigation	4	Forward/reverse transitions and wraparound.
System state	4	ACTIVE/no timeout; ACTIVE/timeout; INACTIVE/no motion; INACTIVE/motion.

Minimum total: 13 meaningful unit tests. Trivial tests that do not exercise decision logic will not satisfy the requirement.

44. Run Unit Tests

```
pio test
```

All final tests must pass. Screenshots do not substitute for test source files committed to the repository.

PART XV — Static Code Analysis**45. Run PlatformIO Check**

```
pio check
```

Students must inspect and interpret findings rather than merely show command output.

46. Static Analysis Findings Table

Finding	File/Line	Cause	Resolution
Example: unused variable	sensors.cpp:xx	Obsolete variable	Removed
Example: conversion warning	display.cpp:xx	Implicit conversion	Type corrected

PART XVI — Functional Verification in Wokwi**47. Required Functional Tests**

ID	Stimulus	Expected Result
FT-01	Change temperature	Displayed temperature updates.
FT-02	Change humidity	Displayed humidity updates.
FT-03	Change light input	Light value changes.
FT-04	Rotate encoder clockwise	Next page is selected.
FT-05	Rotate encoder counterclockwise	Previous page is selected.

FT-06	Set temperature above 30 °C	Alarm activates.
FT-07	Return temperature to normal	Alarm stops.
FT-08	Trigger PIR	System is ACTIVE.
FT-09	Allow inactivity timeout	System becomes INACTIVE.
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE.

48. Verification Record

Test ID	Input/Stimulus	Expected	Actual	Result
FT-01	Example: 28 °C	28 °C displayed	[record observation]	PASS/FAIL

A test shall not be marked PASS without recording the actual observed behavior.

PART XVII — Deliberate FreeRTOS Fault Experiments

49. Fault Experiment 1 — Remove Blocking

Temporarily remove the blocking delay from a continuously executing task. Observe scheduling and responsiveness. Explain CPU usage implications, task starvation risk, and the effect on other tasks. Restore the correct implementation afterward.

50. Fault Experiment 2 — Change Priority

Temporarily assign an unnecessarily high priority to a task that performs frequent work. Observe whether other tasks become less responsive. Explain why. Restore the intended priority afterward.

51. Fault Experiment 3 — Remove Mutex

Temporarily remove Serial protection. Observe whether output becomes interleaved. Explain the result and restore the mutex afterward.

PART XVIII — Git and GitHub

52. Git Is Part of the Laboratory

GitHub is not a final upload location. Commit after meaningful engineering milestones so that the repository documents development history.

- Initialize ESP32 PlatformIO project
- Configure Wokwi simulation
- Add initial FreeRTOS tasks
- Implement DHT22 sensor acquisition
- Add LDR measurement
- Add sensor data queue
- Implement OLED display task
- Add rotary encoder navigation
- Implement alarm task
- Add PIR motion monitoring
- Add system state machine
- Add FreeRTOS event group
- Protect serial output with mutex
- Add alarm unit tests
- Add navigation unit tests
- Add state machine unit tests
- Resolve static analysis findings
- Complete Wokwi verification
- Finalize technical documentation

53. Unacceptable Commit History

- update
- changes
- working
- final
- final2
- finalfinal

A repository containing only one final bulk commit demonstrates little or no development process.

PART XIX — Public README.md

54. Purpose of the README

The README is not the laboratory report. It is the public technical introduction to the portfolio project. Assume the reader is an embedded systems engineer, potential employer, internship evaluator, or another developer.

55. Required README Structure

```
# Project Title

## Project Overview
## Features
## Learning Objectives
## System Architecture
```

```
## FreeRTOS Architecture
## Hardware / Simulated Components
## Pin Configuration
## Task Design
## Inter-Task Communication
## State Machine
## Repository Structure
## Getting Started
## Building the Project
## Running the Wokwi Simulation
## Unit Testing
## Static Code Analysis
## Functional Verification
## Engineering Decisions
## Limitations
## Future Improvements
## References and Acknowledgments
```

56. Required README Visuals

1. Wokwi circuit image.
2. System architecture diagram.
3. FreeRTOS task-communication diagram.
4. State-machine diagram.
5. Finished-system screenshot.

Every figure must support a technical point and use a clear caption. Decorative screenshots do not substitute for technical documentation.

PART XX — Laboratory Report

57. Separate Academic Report

The academic report must be separate from the public README and stored in the repository as:
`docs/laboratory-report.pdf`

The README showcases the project. The report demonstrates technical reasoning and evidence.

58. Required Report Sections

Section	Required Content
1. Problem and Requirements	Explain the problem, the required system behavior, and the Wokwi adaptation.
2. System Architecture and Design	Hardware architecture, software architecture, subsystem decomposition, state machine.
3. FreeRTOS Architecture	Tasks, priorities, periods/events, task states, communication, synchronization.
4. Implementation	Explain significant implementation decisions; do not narrate source code line-by-line.
5. Verification and Testing	Unit tests, Wokwi functional tests, deliberate fault experiments.
6. Static Code Analysis	Findings, severity, interpretation, and corrective actions.
7. Engineering Discussion	Trade-offs, limitations, debugging, problems encountered, alternative approaches.
8. Conclusion	What was learned and what should be improved.

59. Required FreeRTOS Task Table

Task	Responsibility	Trigger / Period	Priority	IPC	Typical Blocked Condition
SensorTask	Measurements	2 s	2	Queue	Delay
DisplayTask	OLED	event/update	1	Queue	Waiting for data
InputTask	Encoder	short periodic/event	3	Notification/queue	Delay/wait
MotionTask	PIR	periodic/event	3	Event group	Delay/wait
AlarmTask	Alarm logic	sensor update	2	Queue	Waiting for data

The table must be accompanied by technical justification, not merely reproduced as a checklist.

PART XXI — Requirements Traceability

60. Requirements Traceability Matrix

Requirement	Implementation	Verification
FR-01	SensorTask	FT-01
FR-05	DisplayTask	FT-01–FT-03
FR-06	InputTask	Unit navigation tests + FT-04/FT-05
FR-07	AlarmTask	Unit alarm tests + FT-06/FT-07
FR-09	MotionTask / StateTask	Unit state tests + FT-09

Every major requirement must be traceable to implementation and verification evidence.

PART XXII — Hackster.io Portfolio Publication

61. Publish Only After Technical Completion

Before creating the Hackster.io post, complete the final technical verification and push the final source code to GitHub.

```
pio run
pio test
pio check
git status
git push
```

62. Purpose of the Hackster.io Post

The Hackster.io article is the public project showcase. It should explain what was built, why it matters, how it works, how FreeRTOS is used, what was learned, and where the complete source code can be found. Add me as a collaborator, Paul Rodolf P. Castor or use my email: paulrodolf.castor@g.msuiit.edu.ph, in your project for BSCA program accreditation purposes.

63. Recommended Hackster.io Sections

- Project Overview
- Motivation
- Features
- Components
- Circuit
- System Architecture
- FreeRTOS Architecture
- How It Works
- Testing and Verification
- Demonstration
- Challenges Encountered
- Lessons Learned
- Limitations

- Future Improvements
- GitHub Repository
- References

PART XXIV — Individual Technical Checkoff

64. Technical Defense

Each student must be prepared for a brief technical defense. A working simulation does not, by itself, prove understanding.

1. Why did you create SensorTask?
2. Why does each task have its assigned priority?
3. What does vTaskDelayUntil() do?
4. What happens to a task while it is delayed?
5. What information crosses your queue?
6. Why did you use a queue rather than unsynchronized global variables?
7. What resource does your mutex protect?
8. Where could a race condition occur?
9. What does your event group or notification represent?
10. Which task owns the OLED, and why?
11. What happens if a high-priority task never blocks?
12. What is the difference between Ready and Blocked?
13. What functionality did your unit tests actually verify?
14. What did static analysis discover?
15. What would differ if this system ran on physical hardware?

PART XXV — Final Submission Checklist

65. Submission Checklist

Category	Requirement
Build	☐ pio run succeeds
FreeRTOS	☐ At least five meaningful tasks
FreeRTOS	☐ Explicit priorities
FreeRTOS	☐ Queue
FreeRTOS	☐ Mutex
FreeRTOS	☐ Event group or task notification
FreeRTOS	☐ vTaskDelayUntil() used appropriately
FreeRTOS	☐ ACTIVE/INACTIVE state machine
Simulation	☐ DHT22, LDR, PIR, rotary encoder, OLED, buzzer
Simulation	☐ Wokwi simulation operational

Testing	☐ At least 13 meaningful unit tests
Testing	☐ pio test succeeds
Testing	☐ Functional verification table completed
Testing	☐ Fault experiments completed
Quality	☐ pio check completed
Quality	☐ Findings analyzed and significant warnings addressed
GitHub	☐ Public repository
GitHub	☐ Meaningful commit history
GitHub	☐ Professional README
GitHub	☐ Architecture diagrams
GitHub	☐ No unnecessary binaries/build artifacts
Report	☐ Laboratory report included
Report	☐ FreeRTOS task table
Report	☐ Requirements traceability matrix
Report	☐ Test results and static-analysis discussion
Report	☐ Limitations documented
Portfolio	☐ Hackster.io article
Portfolio	☐ GitHub link and attribution included

66. Grounds for Rejection or Major Deduction

Submissions may receive severe deductions (i.e., -5 points each) or be returned with 0 points when any of the following apply:

- The project does not compile.
- The Wokwi simulation does not run.
- Most functionality is placed in one task.
- FreeRTOS objects exist only to satisfy the checklist.
- Task priorities cannot be justified.
- A task contains an uncontrolled busy loop.
- Shared resources are accessed unsafely.
- The student cannot explain the queue, mutex, event group, or task notification.
- Unit tests are trivial or unrelated to requirements.
- pio check output is shown without interpretation.

- Screenshots are substituted for analysis.
- The README is incomplete or written merely as an academic worksheet.
- Git history contains only a final bulk commit.
- Architecture diagrams disagree with source code.
- Test results cannot be reproduced.
- The Hackster.io article makes unsupported technical or performance claims.
- External material is used without attribution.
- The student cannot explain the submitted implementation during technical checkoff.

67. Grading System for this laboratory activity

Criterion	Weight
System implementation and Wokwi functionality	15%
FreeRTOS task architecture	15%
Scheduling and priority understanding	10%
IPC and synchronization	10%
Unit testing	10%
Static analysis and code quality	10%
Laboratory report and engineering analysis	10%
Git/GitHub engineering practice	8%
Public README / portfolio quality	7%
Hackster.io technical publication	5%
TOTAL	100%

The technical checkoff functions as an authenticity and understanding gate. Serious inability to explain the submitted FreeRTOS architecture may cap the corresponding FreeRTOS marks even if the simulation appears to work. 0 points will automatically given if you cannot explain your own work.